A forty-nine-variable consistency equation

Positive integer unknowns; exact degree 4; a 185-operation evaluator.

The only unknowns are the forty-nine separately named positive-integer variables

$$J,F,\alpha,f,g,T,X,Y,\xi,\eta,\zeta,\upsilon,\gamma,\vartheta,\psi,\varphi,\epsilon,$$
$$c,a,\ell,h,t,s,x,m,n,p,j,d,o,e,z,\delta,$$
$$\kappa,\lambda,\sigma,\tau,\chi,\theta,\nu,Q,$$
$$N,O,P,S,A,i,w,y\in\mathbb{Z}_{>0}.$$

All other symbols below are polynomial abbreviations:

$$b=\beta J,\quad q=1+b,\quad C=q-F-\alpha,\quad U=Y\kappa,$$
$$E=\lambda,\quad k=\eta+\zeta,\quad W=h-t,$$
$$G=i,\quad \Gamma=w-y,$$
$$H=3+4a,\quad \Delta=\delta,\quad L=P,$$
$$M=sb,$$
$$R=G(\kappa-1)+J(\mu+\omega q),$$
$$Z=p-j,\quad D=d-o,\quad V=e-z,$$
$$I=\theta,$$
$$\mathcal{P}=Z^2-\sigma(1+\tau),$$
$$\mathcal{D}=D^2-\chi(1+N),$$
$$\mathcal{Q}=\nu-Q[\nu-\vartheta]-1,$$
$$\mathcal{R}=I^2-Q,\quad \mathcal{E}=\vartheta-V^2,$$
$$\mathcal{B}=\psi-M,\quad \mathcal{C}=\varphi-U,\quad \mathcal{G}=\epsilon-R,$$
$$\mathcal{H}=c-\eta-O,\quad \mathcal{T}=a-E-Y,$$
$$\mathcal{L}=\ell-EO,\quad \mathcal{W}=W-[C(X+Kq)-Fq],$$
$$\mathcal{X}=s-q\kappa,\quad \mathcal{Y}=x-Eq,$$
$$\mathcal{Z}=m-f^2,\quad \mathcal{N}=n-c^2,$$
$$\mathcal{I}=Z-[\psi(\ell+k+\xi-\Gamma)+Yb-Ws],$$
$$\mathcal{J}=D-[\varphi L+H(S+x+\epsilon+1-k)],$$
$$\mathcal{K}=V-[m(A-\epsilon)-c],\quad \mathcal{O}=\delta-H-a^2,$$
$$\mathfrak{J}=\kappa-q^2,\quad \mathcal{F}=\lambda-XY,$$
$$\mathcal{M}=\sigma-\psi^2,\quad \mathcal{S}=\tau-\ell(\ell+k),$$
$$\mathcal{U}=\chi-\varphi^2,\quad \mathcal{V}=\theta-ng,$$
$$\mathfrak{A}=\nu-\upsilon^2,\quad \mathfrak{B}=Q-\Delta(m-1),$$
$$\mathfrak{C}=N-\delta n,\quad \mathfrak{D}=G-[\alpha+(q-F)b],$$
$$\mathfrak{E}=O-Yk,\quad \mathfrak{F}=\Gamma-kW,$$
$$\mathfrak{G}=P-X-ac,\quad \mathfrak{H}=S-\varphi\gamma,$$
$$\mathfrak{I}=A-Tc.$$

The single Diophantine equation is

$$\mathcal{P}^2+\mathcal{D}^2+\mathcal{Q}^2+\mathcal{R}^2+\mathcal{E}^2+\mathcal{B}^2+\mathcal{C}^2+\mathcal{G}^2$$
$$+\mathcal{H}^2+\mathcal{T}^2+\mathcal{L}^2+\mathcal{W}^2+\mathcal{X}^2+\mathcal{Y}^2+\mathcal{Z}^2+\mathcal{N}^2$$
$$+\mathcal{I}^2+\mathcal{J}^2+\mathcal{K}^2+\mathcal{O}^2+\mathfrak{J}^2+\mathcal{F}^2$$
$$+\mathcal{M}^2+\mathcal{S}^2+\mathcal{U}^2+\mathcal{V}^2+\mathfrak{A}^2+\mathfrak{B}^2$$
$$+\mathfrak{C}^2+\mathfrak{D}^2+\mathfrak{E}^2+\mathfrak{F}^2+\mathfrak{G}^2+\mathfrak{H}^2+\mathfrak{I}^2=0.$$

The four fixed integers. Every index, set, sum, product and floor below defines a constant, not another unknown. Put

$$\rho=2^{47},\quad B=\rho^{2011},\quad \beta=B-1,$$
$$G_m(x)=\sum_{j=0}^{m-1}x^j,\quad \Lambda=\rho^{1005}.$$

$$\mu=\beta-G_{951}(\rho)$$
$$-\rho^{960}G_{26}(\rho)-\rho^{1004},$$
$$\omega=\beta+8G_{917}(\rho)-4+(2^{45}-4)\Lambda$$
$$+\rho^{960}G_9(\rho)+\rho^{987}G_8(\rho)+\rho^{1004},$$
$$K=\rho^{18}+\rho^{1004}+\rho^{2020}$$
$$+2\Lambda\Phi(1/\rho),$$

where

$$\chi_i=1+2^{10}(1+w_i+2^8n_i$$
$$+2^{17}p_i+2^{25}s_i)\quad(0\le i\le 941),$$
$$\pi(0)=0,\quad \pi(i)=1+\sum_{j=1}^{941}\left\lfloor\frac{2^{44}-1+\chi_j-\chi_i}{2^{44}}\right\rfloor\quad(1\le i\le 941).$$

$$\Phi(x)=\sum_{i=0}^{941}\chi_i x^{\pi(i)}$$
$$+2^{10}x^{969}(x^9+2^{17})G_8(2x)$$
$$+2^{18}x^{942}(x^{18}+2^{17})G_9(2x).$$

The rational intermediate in K cancels exactly; all four results are positive integers.

The 942 indexed fixed tiles. Use the actions u_j,v_j,d_j decoded below. For $d=0,1$, let

$$h(q,b)=2q+b+2,\quad \mathcal{S}_d=\{u_j:d_j=d,\ 0\le j<430\},$$
$$r_{d,q}=205-199d+|\mathcal{S}_d\cap\{0,\dots,q-1\}|.$$

An interval ending at -1 is empty. The table defines t_i for $0\le i\le 941$; all index ranges are fixed:

index i	tile $t_i=(w_i,n_i,p_i,s_i)$
0	$(2,435,3,1)$
$j+1$ $(j=2q+b<430)$	$((1-d_j)r_{0,u_j},h(q,b),d_jr_{1,u_j},v_j)$
$433-2a+b$ $(a,b=0,1)$	$(0,432a+b,0,432a+b)$
$\substack{423+2r_{d,q}+b\\ q\in\mathcal{S}_d,\ d,b=0,1}$	$(dr_{d,q},b,(1-d)r_{d,q},h(q,b))$
$923+2a+b$ $(a,b=0,1)$	$(0,432a+b,0,438)$
927	$(2,434,2,0)$
928	$(3,436,4,4)$
929	$(4,436,4,0)$
$496+t$ $(t=434,435,436)$	$(1,438,1,t)$
$933+3a+b$ $(a,b=0,1,2)$	$(a^2,437,b,437)$

A closed decoder for the 430 actions. Put $D_b(A,k)=\left\lfloor A/b^k\right\rfloor \bmod b$, where $a \bmod b = a-b\lfloor a/b\rfloor$. For $0\le q<215$, $b=0,1$, $j=2q+b$:

$$\mathcal{N}=\mathcal{A}\bmod 2^{636},\quad \mathcal{T}=\left\lfloor\mathcal{A}/2^{636}\right\rfloor,$$
$$\nu_j=D_2(\mathcal{N},j),\quad \kappa_j=(\sigma_j+2)^{1-\nu_j}\quad(0\le j<430),$$
$$\sigma_j=\sum_{k=0}^{j-1}\nu_k,\quad \Pi_j=\prod_{k=0}^{j-1}\kappa_k\quad(0\le j\le 430),$$
$$m_q=D_2(\mathcal{N},430+q),\quad \tau_q=\sum_{k=0}^{q-1}m_k,$$
$$F_q=14+m_q(D_{16}(\lfloor\mathcal{T}/\Pi_{430}\rfloor,\tau_q)-14),$$
$$u_j=(\sigma_{j+1}-(\lfloor\mathcal{T}/\Pi_j\rfloor \bmod \kappa_j))\bmod 216,$$
$$v_j=D_2(F_q,2b),\quad d_j=D_2(F_q,2b+1).$$

Empty sums/products are 0/1. Here $(v_0,d_0,u_0)=(1,1,1)$; the constructor tile uses this actual first target, including its state renaming.

The integer $\mathcal{A}$ in base 10. Its complete decimal expansion is the concatenation of these lines:

```
12889260410359561746736540743515845134244260640012702033212621
95398074180784058571518488282119861023093355389406939414042892
97455773992193784795705001618777025966369137954859764532832722
75411843410809038713502633204617333246733616940964578272360901
56186216508504953649192752154519268024509952082950238865468307
22999632482769253980274091217460707829023970817244790340704449
09085581271192071656170732004937337082171141082486384269072884
72778087419663417942892520175630649792329265834490253035577533
96449689808375788457488971017043689875433162972390177936123032
89025657757489526342497438772045714099661532481123050235898973
27443609903800541199351405985754278307227777584454508422717998
33411356440933534607489268032843 7
```

The fixed integer $\mathcal{A}$ above is written in base 10. Lean proves that the formalized equation has no positive solution exactly when the chosen arithmetized ZFC consistency statement holds, by a derivation in the specified intuitionistic PRA. The literal endpoint was independently replayed on a fixed source snapshot; some transitive dependencies remain inherited. The exact degree is certified by integer-polynomial calculations; the 185-operation count is for the unshifted circuit. Neither count is a minimality claim.

Exact 185-operation circuit

Each numbered line defines one gate over the integers. The inputs are the forty-nine unknowns and the four fixed integers above. The ASCII names `alpha, xi, eta, zeta, upsilon, gamma, vartheta, psi, varphi, epsilon, beta, mu, omega, ell, Delta, delta, kappa, lambda, sigma, tau, chi, theta, nu, Gamma` refer to their displayed counterparts; `g` is the unknown g. The last gate is zero exactly when the displayed equation holds. The circuit inputs `cNamed, aNamed, ellNamed` are c, a, ℓ; gate `WNamed` is the displayed W, while gate `W` is $C(X + Kq) - Fq$. The inputs `s, x, m, n, p, j, d, o, e, z, delta` are the displayed $s, x, m, n, p, j, d, o, e, z, \delta$; `kappa, lambda, sigma, tau, chi, theta, nu, Q` are $\kappa, \lambda, \sigma, \tau, \chi, \theta, \nu, Q$; `N, sumY, P, S, A, i, w, y` are N, O, P, S, A, i, w, y. Gates `E, L, I` compute $XY, X+ac, ng$; the display uses $E = \lambda$, $L = P$, $I = \theta$. Intermediate gate names below are local abbreviations, not additional unknowns.

```
1.  b = J * beta
2.  q = 1 + b
3.  q2 = q * q
4.  q3 = q * kappa
5.  U = Y * kappa
6.  qF = q - F
7.  C = qF - alpha
8.  E = X * Y
9.  k = eta + zeta
10.  kY = Y * k
11.  c = eta + sumY
12.  c2 = cNamed * cNamed
13.  a = lambda + Y
14.  g15 = 4 * aNamed
15.  H = 3 + g15
16.  g17 = aNamed * aNamed
17.  DeltaComputed = H + g17
18.  g19 = aNamed * cNamed
19.  L = X + g19
20.  ell = lambda * sumY
21.  M = s * b
22.  g25 = K * q
23.  g26 = X + g25
24.  g27 = C * g26
25.  g28 = F * q
26.  W = g27 - g28
27.  g30 = ellNamed + k
28.  g31 = g30 + xi
29.  WNamed = h - t
30.  root_skewComputed = k * WNamed
31.  root_skew = w - y
32.  root_inner = g31 - root_skew
33.  root_scaled = psi * root_inner
34.  g33 = Y * b
35.  root_btq3 = WNamed * s
36.  root_sum = root_scaled + g33
37.  ZComputed = root_sum - root_btq3
38.  Z = p - j
39.  g38 = Z * Z
40.  g39 = psi * psi
41.  g40 = ellNamed * g30
42.  g41 = 1 + tau
43.  g42 = sigma * g41
44.  A0 = g38 - g42
45.  g44 = qF * b
46.  GComputed = alpha + g44
47.  g46 = kappa - 1
48.  g47 = i * g46
49.  g48 = omega * q
50.  g49 = mu + g48
51.  g50 = J * g49
52.  R = g47 + g50
53.  g52 = k - epsilon
54.  r0 = g52 - 1
55.  g54 = varphi * gamma
56.  g55 = lambda * q
57.  g56 = S + x
58.  shift = g56 - r0
59.  g58 = varphi * P
60.  g59 = H * shift
61.  DstarComputed = g58 + g59
62.  D = d - o
63.  g61 = D * D
64.  g62 = varphi * varphi
65.  g63 = delta * n
66.  g64 = 1 + N
67.  g65 = chi * g64
68.  A1 = g61 - g65
69.  f2 = f * f
70.  rho = m - 1
71.  QComputed = delta * rho
72.  I = n * g
73.  g71 = theta * theta
74.  g72 = g71 - Q
75.  g74 = T * cNamed
76.  g75 = A - epsilon
77.  g76 = m * g75
78.  VComputed = g76 - cNamed
79.  V = e - z
80.  y2 = upsilon * upsilon
81.  g80 = V * V
82.  g81 = nu - vartheta
83.  g82 = Q * g81
84.  Fa = nu - g82
85.  auxResidual = Fa - 1
86.  firstSquare = A0 * A0
87.  mainSquare = A1 * A1
88.  auxSquare = auxResidual * auxResidual
89.  strongSquare = g72 * g72
90.  firstMain = firstSquare + mainSquare
91.  firstMainAux = firstMain + auxSquare
92.  sumSquares = firstMainAux + strongSquare
93.  difference = vartheta - g80
94.  constraint = difference * difference
95.  result = sumSquares + constraint
96.  scaleDiff = psi - M
97.  scaleCheck = scaleDiff * scaleDiff
98.  final = result + scaleCheck
99.  mainDiff = varphi - U
100.  mainCheck = mainDiff * mainDiff
101.  result16 = final + mainCheck
102.  rankDiff = epsilon - R
103.  rankCheck = rankDiff * rankDiff
104.  result17 = result16 + rankCheck
105.  cDifference = cNamed - c
106.  cConstraint = cDifference * cDifference
107.  aDifference = aNamed - a
108.  aConstraint = aDifference * aDifference
109.  ellDifference = ellNamed - ell
110.  ellConstraint = ellDifference * ellDifference
111.  wDifference = WNamed - W
112.  wConstraint = wDifference * wDifference
113.  addC = result17 + cConstraint
114.  addA = addC + aConstraint
115.  addEll = addA + ellConstraint
116.  finalWithNamedValues = addEll + wConstraint
117.  cubeDifference = s - q3
118.  cubeConstraint = cubeDifference * cubeDifference
119.  productDifference = x - g55
120.  productConstraint = productDifference *
productDifference
121.  fSquareDifference = m - f2
122.  fSquareConstraint = fSquareDifference *
fSquareDifference
123.  cSquareDifference = n - c2
124.  cSquareConstraint = cSquareDifference *
cSquareDifference
125.  addCube = finalWithNamedValues + cubeConstraint
126.  addProduct = addCube + productConstraint
127.  addFSquare = addProduct + fSquareConstraint
128.  addCSquare = addFSquare + cSquareConstraint
129.  zDifference = Z - ZComputed
130.  zConstraint = zDifference * zDifference
131.  dDifference = D - DstarComputed
132.  dConstraint = dDifference * dDifference
133.  vDifference = V - VComputed
134.  vConstraint = vDifference * vDifference
135.  deltaDifference = delta - DeltaComputed
136.  deltaConstraint = deltaDifference *
deltaDifference
137.  addZConstraint = addCSquare + zConstraint
138.  addDConstraint = addZConstraint + dConstraint
139.  addVConstraint = addDConstraint + vConstraint
140.  addDELTAConstraint = addVConstraint +
deltaConstraint
141.  qSquareDifference = kappa - q2
142.  qSquareConstraint = qSquareDifference *
qSquareDifference
143.  product2Difference = lambda - E
144.  product2Constraint = product2Difference *
product2Difference
145.  psiSquareDifference = sigma - g39
146.  psiSquareConstraint = psiSquareDifference *
psiSquareDifference
147.  ellProductDifference = tau - g40
148.  ellProductConstraint = ellProductDifference *
ellProductDifference
149.  varphiSquareDifference = chi - g62
150.  varphiSquareConstraint = varphiSquareDifference *
varphiSquareDifference
151.  scaledIndexDifference = theta - I
152.  scaledIndexConstraint = scaledIndexDifference *
scaledIndexDifference
153.  ySquareDifference = nu - y2
154.  ySquareConstraint = ySquareDifference *
ySquareDifference
155.  qSignedDifference = Q - QComputed
156.  qSignedConstraint = qSignedDifference *
qSignedDifference
157.  add_qSquare = addDELTAConstraint +
qSquareConstraint
158.  add_product2 = add_qSquare + product2Constraint
159.  add_psiSquare = add_product2 + psiSquareConstraint
160.  add_ellProduct = add_psiSquare +
ellProductConstraint
161.  add_varphiSquare = add_ellProduct +
varphiSquareConstraint
162.  add_scaledIndex = add_varphiSquare +
scaledIndexConstraint
163.  add_ySquare = add_scaledIndex + ySquareConstraint
164.  add_qSigned = add_ySquare + qSignedConstraint
165.  deltaProductDifference = N - g63
166.  deltaProductConstraint = deltaProductDifference *
deltaProductDifference
167.  rankSeedDifference = i - GComputed
168.  rankSeedConstraint = rankSeedDifference *
rankSeedDifference
169.  ySumDifference = sumY - kY
170.  ySumConstraint = ySumDifference * ySumDifference
171.  rootSkewDifference = root_skew - root_skewComputed
172.  rootSkewConstraint = rootSkewDifference *
rootSkewDifference
173.  offsetDifference = P - L
174.  offsetConstraint = offsetDifference *
offsetDifference
175.  gammaProductDifference = S - g54
176.  gammaProductConstraint = gammaProductDifference *
gammaProductDifference
177.  timeProductDifference = A - g74
178.  timeProductConstraint = timeProductDifference *
timeProductDifference
179.  add_deltaProduct = add_qSigned +
deltaProductConstraint
180.  add_rankSeed = add_deltaProduct +
rankSeedConstraint
181.  add_ySum = add_rankSeed + ySumConstraint
182.  add_rootSkew = add_ySum + rootSkewConstraint
183.  add_offset = add_rootSkew + offsetConstraint
184.  add_gammaProduct = add_offset +
gammaProductConstraint
185.  add_timeProduct = add_gammaProduct +
timeProductConstraint
```